Chapter 1 worked solutions

Tokens, Embeddings, and Neural Language Models

These written solutions cover Exercises 1–7 and 9. The companion notebook and its PDF cover Exercises 8 and 10, which use the Chapter 1 Python model. The examples below use the chapter's word-level vocabulary, zero-based IDs, and hand-chosen weights unless the exercise specifies a different tokenizer or gives its own probabilities.

1. From tokens to IDs to embeddings

Start with the text *the red car*. The chapter's tokenizer treats each word in this phrase as one token. Read the tokens in order, find their IDs in the vocabulary, and use those IDs to retrieve the rows of the embedding table E.

Position in the phrase	Token	ID	Row of E
First	*the*	4	$(0.2, -0.4, 0.1, 0.5)$
Second	*red*	12	$(-0.3, 0.8, 0.2, -0.1)$
Third	*car*	7	$(0.6, 0.1, -0.5, 0.3)$

The ID 12 tells us which vocabulary entry represents *red*. Because the table uses zero-based indexing, we retrieve $E[12, :]$, the row with index 12, taking all four columns. The single integer 12 serves as an address. The four real numbers in that row are the vector the model uses in its calculations. For instance, the difference $12 - 7 = 5$ tells us how far apart the two row indices are. Any comparison of the two embeddings would have to use their vector coordinates.

There are 16 vocabulary entries, each with a four-coordinate embedding, so E has shape $(16, 4)$. The phrase uses three token positions. Stack the rows selected by $(4, 12, 7)$ in that order to obtain

$$X = \begin{bmatrix} 0.2 & -0.4 & 0.1 & 0.5 \\ -0.3 & 0.8 & 0.2 & -0.1 \\ 0.6 & 0.1 & -0.5 & 0.3 \end{bmatrix}.$$

The result has shape $(3, 4)$. A row of E belongs to a **vocabulary entry**. A row of X belongs to a **position in this phrase**. In particular, the first row of X comes from row 4 of E, since the first token is *the*.

2. Looking up a repeated token

Read *red car red* from left to right. The ID sequence is $(12, 7, 12)$, so the lookup reads $E[12, :]$, then $E[7, :]$, then $E[12, :]$ again. The three retrieved rows are

$$X = \begin{bmatrix} -0.3 & 0.8 & 0.2 & -0.1 \\ 0.6 & 0.1 & -0.5 & 0.3 \\ -0.3 & 0.8 & 0.2 & -0.1 \end{bmatrix}.$$

The first and third rows agree coordinate by coordinate. Both positions hold the same ID, so both retrieve the same vector. The middle row comes from ID 7 and holds the vector for *car*. There are three rows even though the phrase uses only two distinct vocabulary entries. The matrix records every occurrence in its original position.

The first *red* is before *car*, while the second *red* is after it. The lookup makes the same decision at either position because it uses only the ID at that position. More generally, if $k_i = k_j$, then $E[k_i, :] = E[k_j, :]$ for that table. The rest of the phrase plays no role in that equality.

A later computation can combine vectors from neighboring positions. For instance, Exercise 6 shows how the two-token network weights the first and second positions differently. The input vectors still come from the same shared table. What changes is how the network combines them.

3. Renumbering the vocabulary consistently

Give *the* its new ID 12 and *red* its new ID 4. The text *the red car* is now encoded as $(12, 4, 7)$. To keep the same vectors at those three positions, move each embedding with its word. If E' is the new table, the affected rows are

$$E'[12, :] = E[4, :], \qquad E'[4, :] = E[12, :].$$

Every other row stays in its original position. The new sequence retrieves the *the* vector from row 12, the *red* vector from row 4, and the *car* vector from row 7. This reproduces the same X as in Exercise 1. Updating only the IDs would leave the old *red* vector at row 12 and the old *the* vector at row 4. The model would then receive the embeddings of *red the car* in that order.

The model also uses IDs to label its output scores. In the chapter's row-vector convention, column j of W_{out} and entry j of $\mathbf{b}_{out}$ together produce the logit for class j. Move both parts of the *the* class from 4 to 12 and both parts of the *red* class from 12 to 4. Thus we swap **rows** 4 and 12 of E, **columns** 4 and 12 of W_{out}, and **entries** 4 and 12 of $\mathbf{b}_{out}$. The target IDs in training data and the mapping used to decode predictions must follow the same numbering.

After the swap, the model has the same hidden vector for a given text because its input embeddings are unchanged. The two relevant output logits exchange slots. Softmax adds the same exponentials in its denominator, so the probabilities exchange slots in the same way. A student reading probabilities by token name gets the original answers. For example, a *red* probability of about 0.112757 moves from index 12 to index 4, while the *car* probability remains at index 7.

4. Counting tokens under two tokenizers

Both tokenizers see the same three written words in *the unhelpful car*. The word-level rule uses one token per word. The teaching subword rule divides *unhelpful* into three pieces and keeps *the* and *car* whole.

Rule	Tokens in order	Count
Word level	*the*, *unhelpful*, *car*	3
Teaching subwords	*the*, *un*, *help*, *ful*, *car*	5

The subword split replaces one token with three, adding two positions. The related split of *helpful* into *help* and *ful* shows how a vocabulary can reuse pieces across different words. Since *helpful* occurs only as a related example, it adds no positions to this phrase.

Byte-pair encoding, or BPE, learns a sequence of merge rules from training text. The procedure starts from small units, such as characters or bytes, and repeatedly adds a merge for an adjacent pair that occurs frequently under the training setup. When tokenizing new text, the learned merges determine which neighboring units can be joined. The training corpus, the starting units, and the number of merges all affect the pieces that are available. For example, if the rules merge *help* with *ful* but do not join *un* with *helpful*, *unhelpful* could become two pieces, *un* and *helpful*.

The teaching rule gives an exact answer here because the exercise supplies the split. To count tokens for a real model, use the tokenizer associated with that model. The same three written words can occupy different numbers of input positions under different tokenizers.

5. Normalizing all sixteen logits

After *the red*, the example gives *car* logit 2, *red* logit 1, and each of the other 14 vocabulary entries logit 0. A logit is an unnormalized score. To apply softmax, first exponentiate each score. The three possible exponentials are $e^2 \approx 7.389056$, $e^1 \approx 2.718282$, and $e^0 = 1$. In particular, a zero logit contributes 1 to the denominator.

Add the contributions of all 16 entries. There are 14 copies of 1, so the denominator is

$$D = e^2 + e^1 + 14e^0 \approx 24.1073379274.$$

Divide each token's exponential by D. Use the full denominator even when calculating only one token's probability.

Vocabulary entry	Probability as a fraction	Approximate probability
car	e^2/D	0.306506514
red	e^1/D	0.112757445
Each of the other 14 entries	$1/D$	0.041481146

To check normalization, count the final row of the table 14 times. Using exact expressions avoids a small discrepancy from rounding the displayed decimals.

$$\frac{e^2}{D} + \frac{e^1}{D} + 14\frac{1}{D} = \frac{e^2 + e^1 + 14}{D} = 1.$$

The two leading tokens carry about 0.419264 of the probability together. The remaining mass, about 0.580736, is spread evenly across the other 14 entries. Although *car* has the largest individual probability, the probability that the next token belongs to that group is larger than the probability of *car*. Dividing by $e^2 + e^1$ would calculate the probability of each of the two leading tokens **given** that the next token is either *car* or *red*.

6. Tracking the effect of token order

The network combines two embeddings into a four-coordinate hidden vector. At each coordinate it takes the first token's value once, adds the second token's value twice, and adds the bias at that coordinate. The bias is $(0, -0.2, 0, 0)$, so it changes only the second coordinate. The pre-activation is the vector obtained after this weighted sum. Applying $\text{ReLU}(a) = \max(0, a)$ separately to each coordinate gives the hidden vector $\mathbf{h}$.

With *the* first and *red* second, the calculation proceeds coordinate by coordinate. The table counts coordinates from 1.

Coordinate	Pre-activation calculation	Before ReLU	After ReLU
1	$0.2 + 2(-0.3) + 0$	-0.4	0
2	$-0.4 + 2(0.8) - 0.2$	1.0	1.0
3	$0.1 + 2(0.2) + 0$	0.5	0.5
4	$0.5 + 2(-0.1) + 0$	0.3	0.3

The hidden vector is $(0, 1.0, 0.5, 0.3)$. Reverse the tokens, keeping the position weights and bias the same. The coefficient 2 now multiplies the embedding for *the*.

Coordinate	Pre-activation calculation	Before ReLU	After ReLU
1	$-0.3 + 2(0.2) + 0$	0.1	0.1
2	$0.8 + 2(-0.4) - 0.2$	-0.2	0
3	$0.2 + 2(0.1) + 0$	0.4	0.4
4	$-0.1 + 2(0.5) + 0$	0.9	0.9

This gives $\mathbf{h} = (0.1, 0, 0.4, 0.9)$. To see which differences affect the scores, inspect the output weights. Only the **second hidden coordinate** contributes to the logits. Multiply it by 2 for *car* and by 1 for *red*. Every other output weight and all output biases are zero.

- After *the red*, the second coordinate is 1. The logits are 2 for *car*, 1 for *red*, and 0 elsewhere. Exercise 5 gives the resulting probabilities, with about 0.306507 for *car* and 0.112757 for *red*.
- After *red the*, the second coordinate is 0. Every logit is 0, even though several other hidden coordinates are positive. Every exponential is then 1, and softmax gives every entry $1/16 = 0.0625$.

The token order changes which embedding receives the larger coefficient. In this example it takes the second pre-activation from above zero to below zero. ReLU then sets that coordinate to zero, which removes the only contribution to the logits. This explains the change from unequal probabilities to a uniform distribution with the same set of tokens and the same weights.

7. Reading the batch axes

A batch groups several examples for the same computation. Here there are five sentences and each has three token positions. Store one integer per position, so the ID array has shape $(5, 3)$. Looking up an ID retrieves four coordinates. NumPy keeps both axes of the ID array and appends the embedding axis, giving shape $(5, 3, 4)$. For example, selecting one sentence from that result gives a $(3, 4)$ matrix, as in Exercise 1.

For the context model, take one two-token window per batch item. Its input ID array has shape $(5, 2)$ and lookup returns shape $(5, 2, 4)$. Join the two four-coordinate embeddings within each item, preserving their order. The first four values are from the first position and the next four are from the second position. This gives five vectors of width 8.

Array	Shape	Meaning of the axes
Sentence IDs	$(5, 3)$	batch item, sentence position
Sentence embeddings	$(5, 3, 4)$	batch item, sentence position, embedding coordinate
Context IDs	$(5, 2)$	batch item, context position
Context embeddings	$(5, 2, 4)$	batch item, context position, embedding coordinate
Joined context vectors	$(5, 8)$	batch item, joined coordinate
Hidden vectors	$(5, 4)$	batch item, hidden coordinate
Logits	$(5, 16)$	batch item, vocabulary entry

The first matrix multiplication maps each eight-coordinate row to four coordinates using the same $(8, 4)$ weight matrix. Adding the four-entry bias and applying ReLU preserves shape $(5, 4)$. The output layer then uses a $(4, 16)$ weight matrix and a 16-entry bias, giving $(5, 16)$. As a check, the five rows persist through both matrix multiplications. Each of those rows contains the 16 alternatives for a single prediction, so softmax normalizes across the final axis separately for each row.

9. Multiplying the probabilities along one path

We want the probability of producing exactly *the red car* and then stopping. Starting from *BOS*, the model has to choose *the* first, then *red* given that *the* was chosen, then *car* given those first two words, and finally *EOS* given the three words. Each factor refers to the probability of taking the next step **given that the path has reached it**.

Step	Prefix already produced	Required next token	Probability
1	*BOS*	*the*	0.5
2	*BOS the*	*red*	0.25
3	*BOS the red*	*car*	0.8
4	*BOS the red car*	*EOS*	0.5

By the probability chain rule, the probability of reaching the second word on this path is $0.5 \times 0.25 = 0.125$. Reaching the third word brings it to $0.125 \times 0.8 = 0.1$. The ending supplies one more factor, giving

$$P(\text{the red car and then EOS} \mid BOS) = 0.5 \times 0.25 \times 0.8 \times 0.5 = 0.05.$$

The exact sentence with its ending therefore has probability **5%**. The intermediate result 0.1 is the probability of starting with *the red car*. The final factor selects the continuations in which the model stops immediately after *car*. The chain rule uses conditional probabilities, so the chance of *car* can depend on the words that precede it.

Adding the four supplied values gives 2.05. This is allowed because the values come from four different vocabulary rows. At the third step, for instance, *car* has probability 0.8 and **all the other vocabulary entries at that same step** share the remaining 0.2. It is those alternatives for a common prefix that sum to one. The four probabilities in the table select one alternative at each of four steps.